

Task delivered. Where did the budget go?

Understand how agents spend tokens, compute and time. Find the bottleneck and check whether the result was worth the resources.

PATH TO DELIVERY

01 → 04

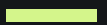
01 Understand the task



02 Make the changes



03 Wait for the test agent



↺ Check again



04 Deliver the result



Illustrative workflow. Bar lengths do not represent measured shares or durations.

The total hides the bottleneck.

A token summary tells us how much was used. People need to know which step is slowing delivery and what they can change.

01

Expensive waiting

The lead agent keeps calling its model while waiting for a smaller agent to finish.

02

Checks on repeat

Tests run again. We need to see whether a change, failure or new requirement justified the next run.

03

Spend without an outcome

High resource use alone says little about success. The task may still be unfinished.

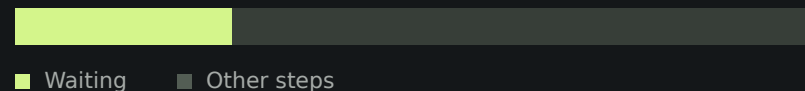
One workflow. Over a billion tokens.

26.9% of this family's recorded tokens belong to responses invoking wait tools. That is 295.1M tokens around waiting.

This covers the lead and its connected subagents in the selected workspace. It includes agent waits, sleep calls and continuations of running tools; it does not attribute every wait to one test agent.

1.10_B

tokens in one lead-agent family



Wait-associated tokens

295.1M

Connected sessions

20

From Codex logs Input + output, including cached input. Most input is cached. Token share is not cost share. The wait itself does not generate model tokens.

One check agent. 300 million tokens.

75.4% of this agent's 300.8M tokens belong to responses that only invoke waits, process polls or coordination messages.

226.9M

tokens around observation

Waits, direct polls and coordination messages. These responses may include useful analysis; the total is a signal to investigate.

1,051

tool-wait continuations

`wait` resumes a yielded execution. Another 162 responses each invoke one direct process poll.

345

coordination messages

`send_message` replies account for 49.7M tokens. Compare their frequency with new information and outcomes.

PATTERNS TO DETECT Frequent polling · redundant updates · reruns without a new reason

Spot the pattern. Explain the cause.

SIGNAL	EVIDENCE TO CHECK	POSSIBLE CHANGE
Waiting with an expensive model	Repeated wait-invoking responses, agent role, context size and cached input.	Use completion events. Do independent work while tests run.
A test already passed	The same check, code and environment, with a previous passing result.	Reuse the result when the relevant inputs have not changed.
Too much observation	Polling and update frequency, new information between calls, time to delivery.	Increase the interval. Report state changes and completion.

A signal starts a review. A rerun after a code change, failure or flaky test may be justified.

One step. Several dimensions.

T

Tokens

Input, cached input and output separately. Model, agent role and specific step.

\$

Cost

Model pricing on the request date. Keep estimates separate from actual billing.

↻

Compute

CPU, memory and test processes. Process elapsed time separately from CPU time.

→

Delivery

Time to outcome, waits and blockers. Task success and the checks that confirm it.

Task → Agent → Model step / tool → Usage + outcome

Cached tokens are part of input. Reasoning tokens are part of output. Avoid double counting and keep tokens separate from dollars.

Everyday agent tools. A shared view of resources.

PROJECT GOAL

Common agent tools

Codex, Claude Code and OpenCode.
Evaluate adapters to their logs, usage
records and tool events.

Follow the tools people already use

PROJECT GOAL

Ordinary environments

Local repositories, terminal sessions
and standard devcontainers. Collect
resource use where the work happens.

Work without Subyard

PROJECT GOAL

Comparable metrics

Connect steps to tokens, compute,
time and outcomes. Make bottlenecks
understandable across workflows.

Simple, clear and reliable

This is a general solution for most ordinary agent workflows. Subyard is one reference environment and a source of examples; the intended solution works independently of it.

Less guesswork. More useful work.

People should quickly see where the budget goes, why delivery is delayed and which change could improve the next run.

01 What was delivered? 02 Where did resources go? 03 What should change?